CitiusTech

Powering the Future of Healthcare

Design Patterns for Microservices Architecture

Version 1.0
January 2024

This document is confidential and contains proprietary information, including trade secrets of CitiusTech. Neither the document nor any of the information contained in it may be reproduced or disclosed to any unauthorized person under any circumstances without the express written permission of CitiusTech.

© 2021 CitiusTech | Private & Confidential

Agenda

- **Takeaway for Various Stakeholders**
- History of Service Based Architectures
- About Microservices
- Microservices Core/Advanced Concepts
- Solving Microservices Problems with Patterns
- Microservices Architecture Choices

Microservices

"Do more with fewer resources."

Takeaway for Various Stakeholders

Managers

- Focus on key benefits
- Explain the risks
- Bottom-line Impact

Architects

- Foundational Knowledge
- Prepare you to ask the right questions
- Expose you to the rewards and risks

Developers

- Foundational Knowledge
- Answer the why questions
- Not a coding course

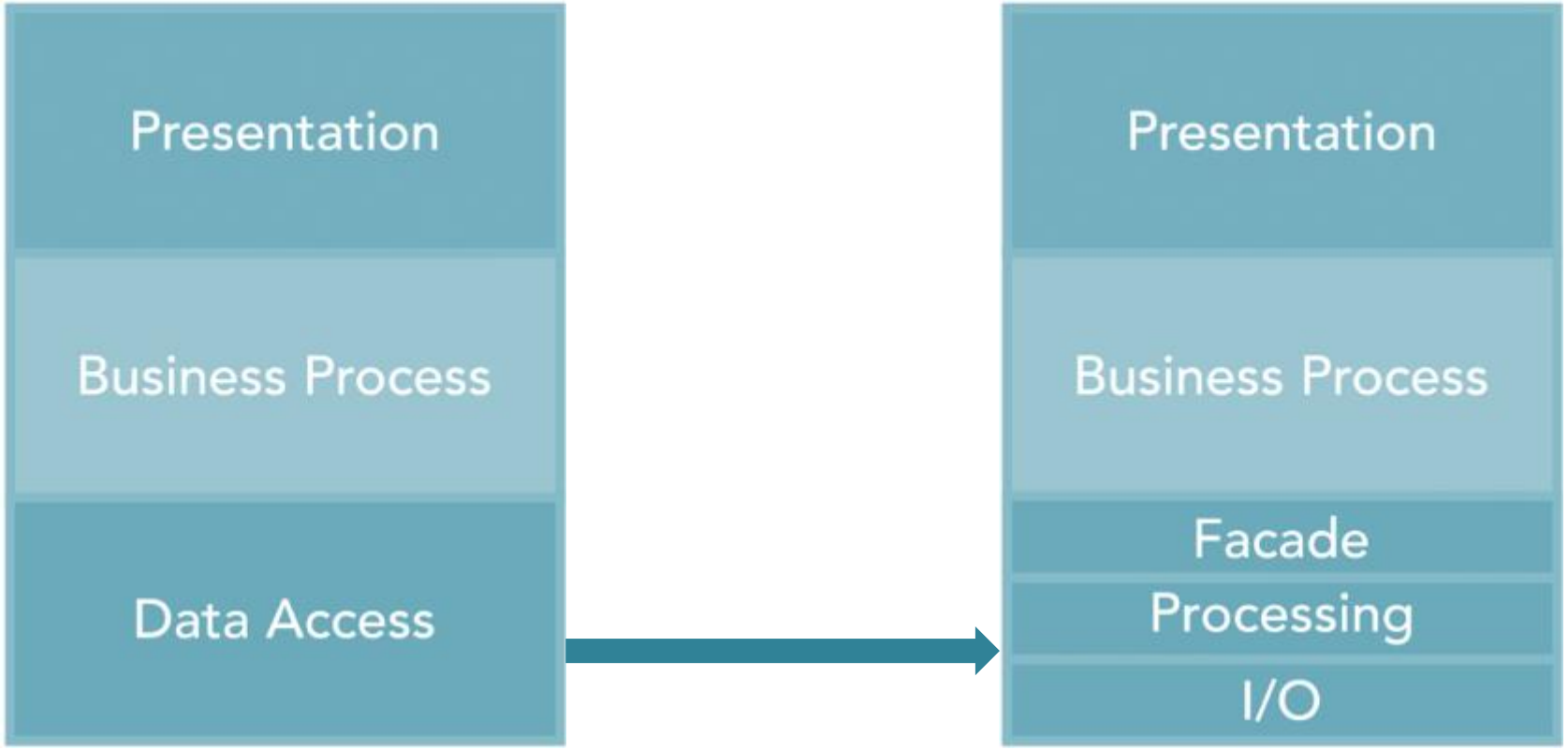
****Non-technical manager may not understand all of the ins and outs of the pattern, there's a clear value in understanding the basics so you can properly empower your teams to do the work needed.

Agenda

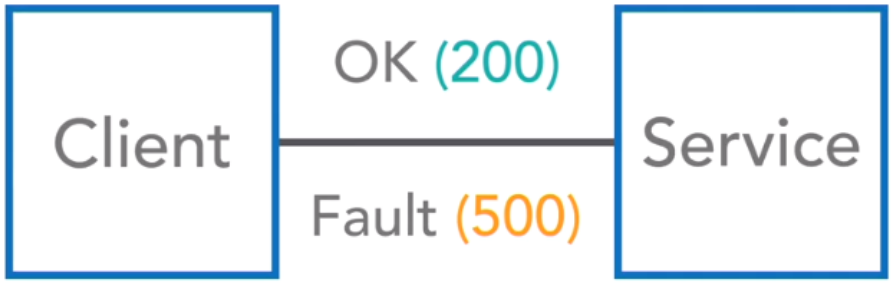
- Takeaway for Various Stakeholders
- **History of Service Based Architectures**
- About Microservices
- Microservices Core/Advanced Concepts
- Solving Microservices Problems with Patterns
- Microservices Architecture Choices

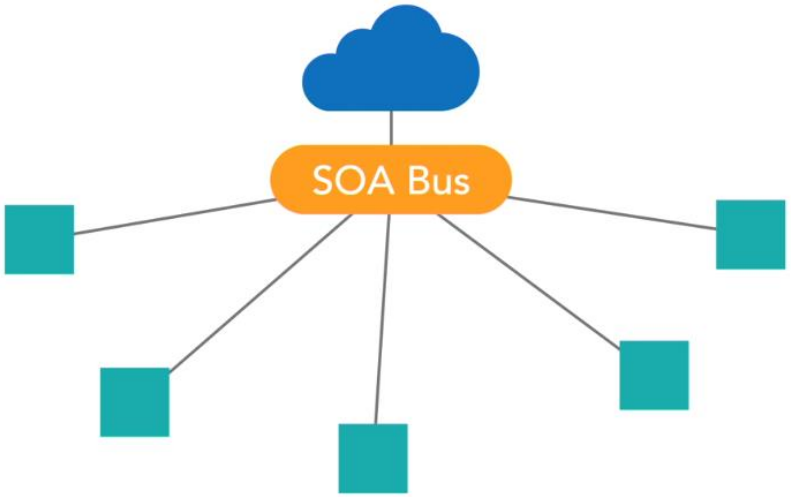
N-tier Monolithic Architecture

- The coupling of code in monoliths make changes hard and the older the application was, the more coupling would develop.
- In addition to the coupling issues, monolithic applications require considerable time to build, test, and deploy.



Service Bases Architecture – SOA



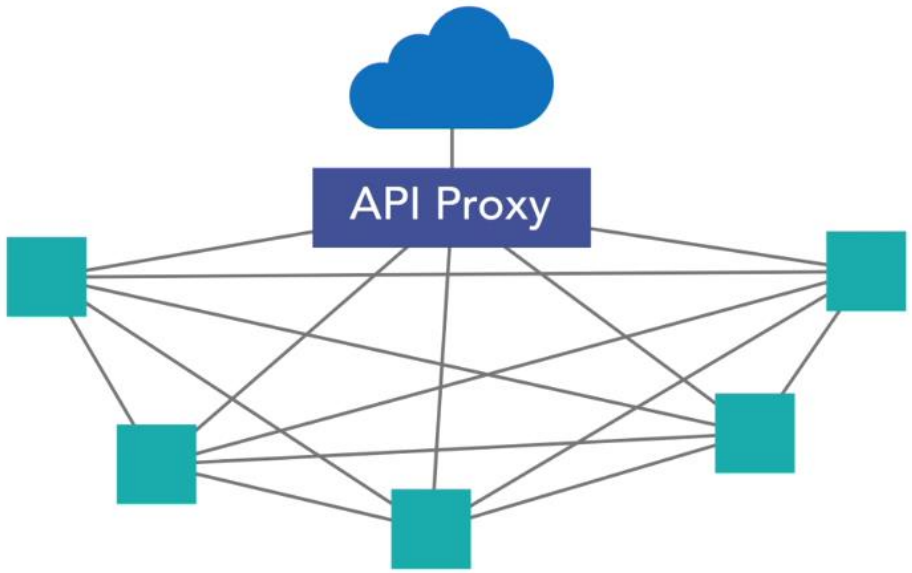


- While SOA decomposed our applications into smaller modules, it brought about two new issues:
  - One is the web service technology was based on soap, and while not a deal-breaker on its own, it makes a lot of compromises to how the web works. Every request for instance is not either okay or an internal server error. There are recoverable errors that can occur.
  - The second and more prevalent issue is the aggregation layer itself. While the aggregation layer can be written very thin, the reality is that transformations of XML and logic operations start getting added to the bus itself. This added a new level of coupling, now between internal and external elements of the system as a whole.
- SOA was however the hottest architecture pattern of the time.

Agenda

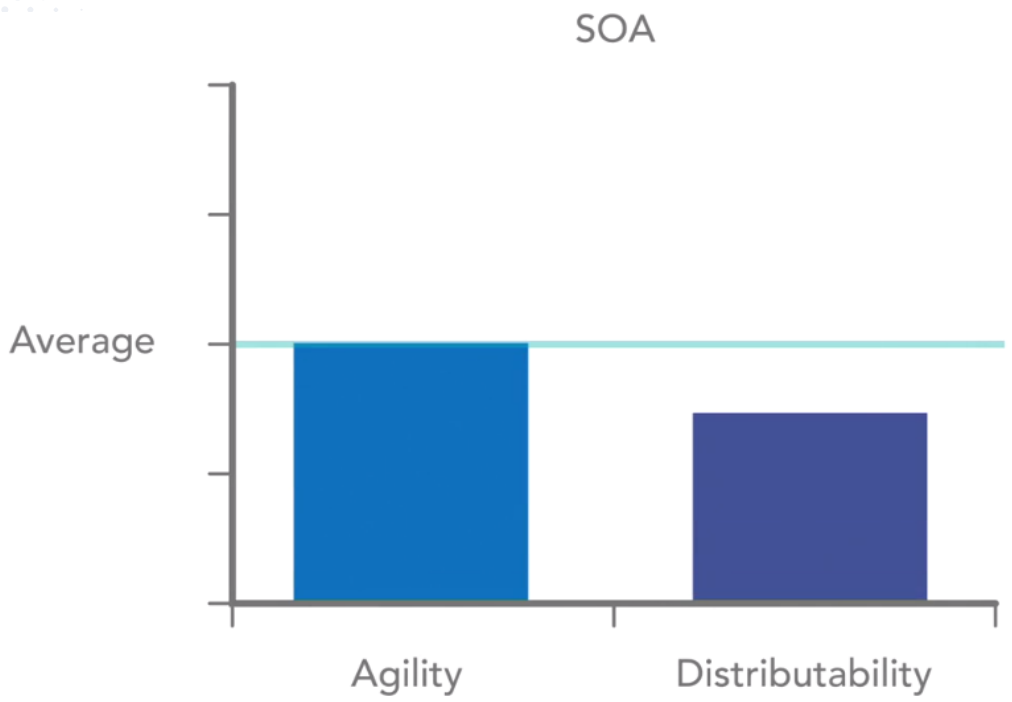
- Takeaway for Various Stakeholders
- History of Service Based Architectures
- **About Microservices**
- Microservices Core/Advanced Concepts
- Solving Microservices Problems with Patterns
- Microservices Architecture Choices

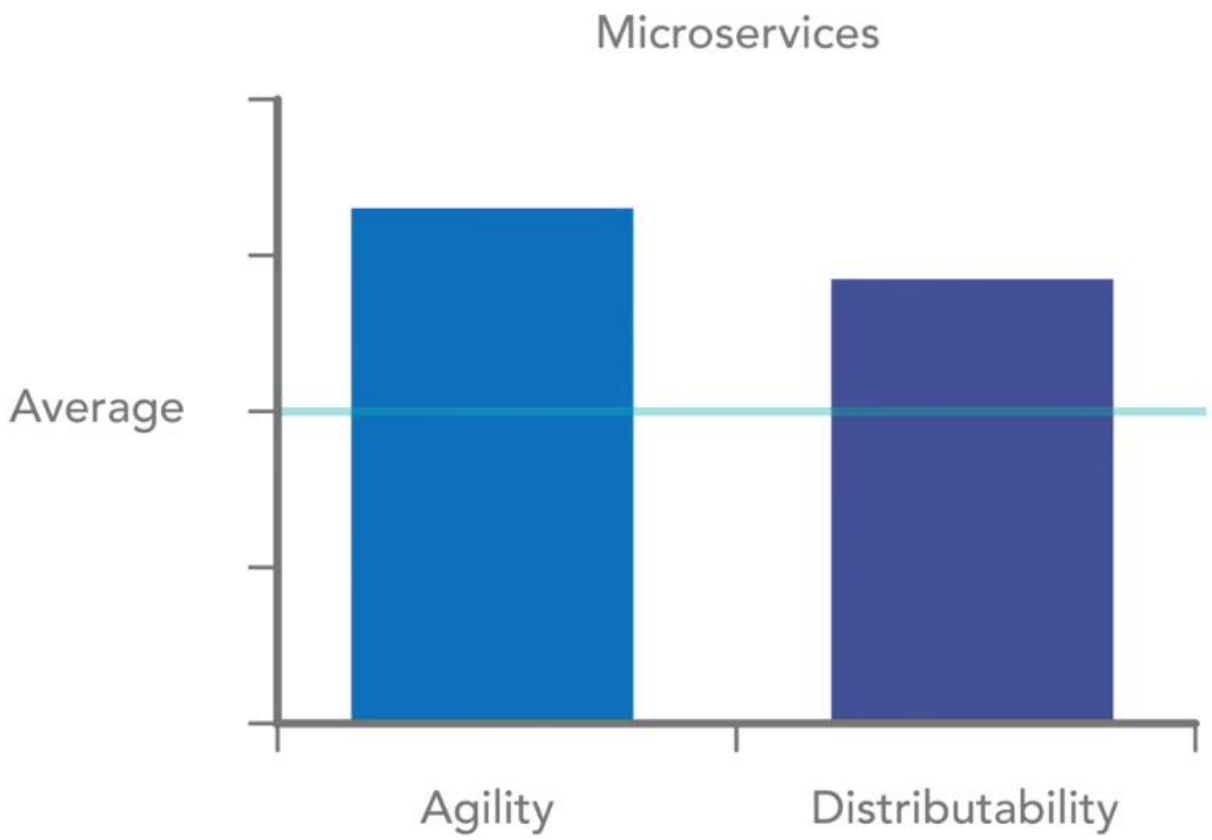
Microservices (1/2)



- Would be the first to state that microservices-based architectures are not the silver bullet.
- In fact, they bring their own sets of problems.
- They do however have the promise of a more agile framework that can be extended into a cloud native world much easier than either monolithic applications or SOA-based architectures.
- The model fits well in both a web developer and web service developer, in part because HTTP is at the heart of the model.

Microservices (2/2)



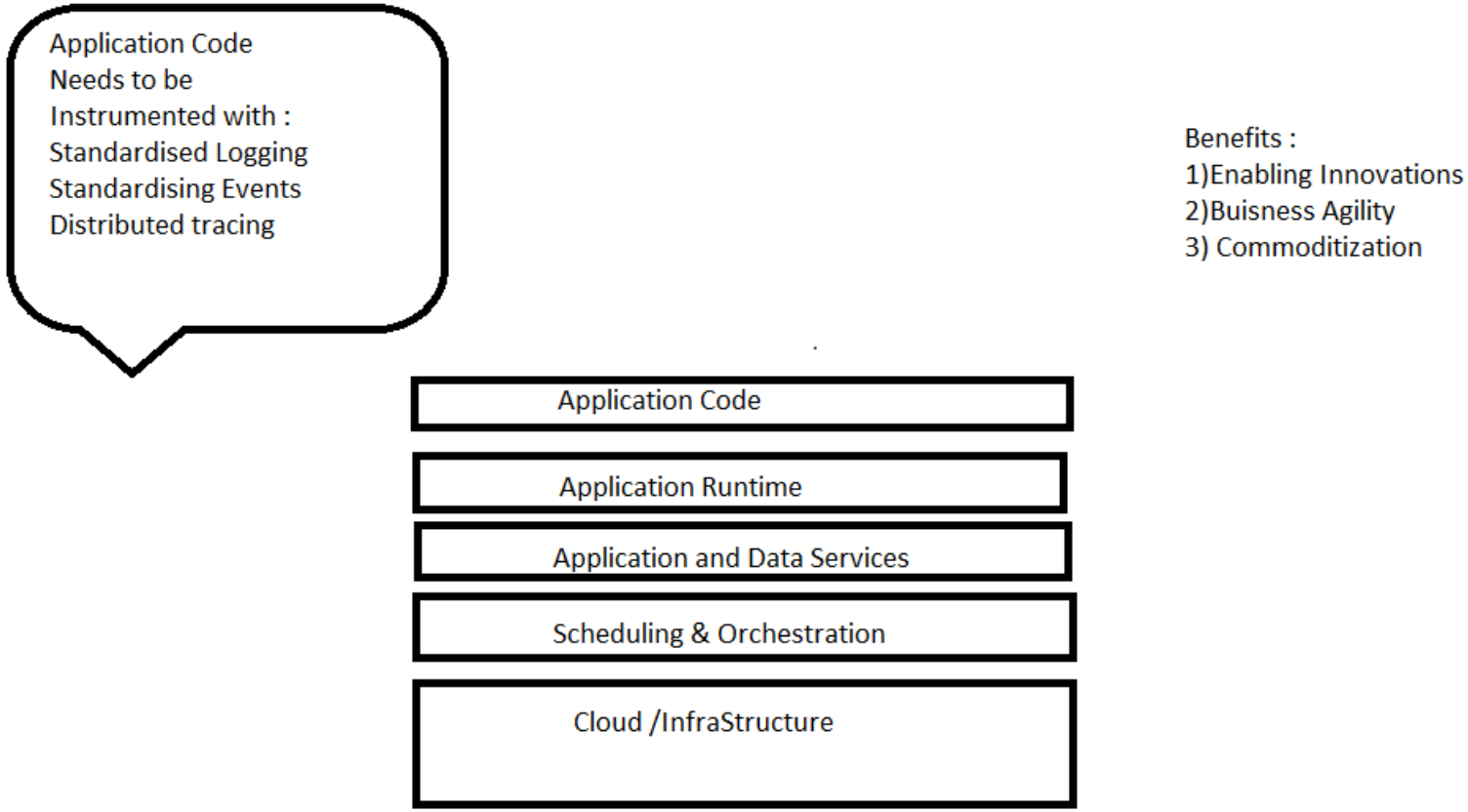


Agenda

- Takeaway for Various Stakeholders
- History of Service Based Architectures
- About Microservices
- **Microservices Core/Advanced Concepts**
- Solving Microservices Problems with Patterns
- Microservices Architecture Choices

Cloud Native Apps

- In general usage, “cloud-native” is an approach to building and running applications that exploits the advantages of the cloud computing delivery model. “Cloud-native” is about how applications are created and deployed, not where.
- It implies that the apps live in the public cloud, as opposed to an on-premises datacenter.



Cloud Native Apps Key Features

- Packaged as lightweight containers.
- Developed with best-of-breed languages and frameworks.
- Designed as loosely coupled microservices.
- Centered around APIs for interaction and collaboration.
- Architected with a clean separation of stateless and stateful services.
- Isolated from server and operating system dependencies.
- Deployed on self-service, elastic, cloud infrastructure.
- Managed through agile DevOps processes.
- Automated capabilities:(IaaC).
- Defined, policy-driven resource allocation: Central IT.
- Developers and DevOps teams in each department have complete access and ownership to their share of resources.

12-Factor Application (1/2)

- The 12-factor app is a methodology defined by the developers at Heroku for building cloud-native applications that:
  - Use declarative formats for setup automation, which minimizes time and cost for new developers joining the project.
  - Have a clean contract with the underlying operating system, offering maximum portability between execution environments.
  - Are suitable for deployment on modern cloud platforms.
  - Minimize divergence between development and production, enabling continuous deployment for maximum agility.
  - Can scale up without significant changes to tooling, architecture, or development practices.

12-Factor Application (2/2)

The Original 12 Factors from this Methodology are:

- **Codebase** - One codebase tracked in revision control; many deploys.
- **Dependencies** - Explicitly declare and isolate dependencies.
- **Configuration** - Store configuration in the environment.
- **Backing services** - Treat backing services as attached resources.
- **Build, release, run** - Strictly separate build and run stages.
- **Processes** - Execute the app as one or more stateless processes.
- **Port binding** - Export services via port binding.
- **Concurrency** - Scale-out via the process model.
- **Disposability** - Maximize robustness with fast start-up and graceful shutdown.
- **Dev/prod parity** - Keep development, staging, and production as similar as possible.
- **Logs** - Treat logs as event streams.
- **Admin processes** - Run admin/management tasks as one-off processes.

Vernacular of Microservices

Service Types

- **Data Service.**
- **Business Services**(Order Service needs order and Customer Data).
- **Translation Service**(Wrapper on Third party services):
  - Its abstraction on a third-party operation that you want to encapsulate under your own façade.
- **Edge Service**(Responsible for serving data to users and external services) - Mobile view, WebView.

Platforms

- **Runtime for Services**(This can include bare metal servers, virtualization through VMs, either public or private, or containerized runtimes like Kubernetes or Cloud Foundry).
- **Ancillary Services** - We don't write but included in platform (Message Queue, Cash Services, Authentication/Auth Services).
- **Operational Components** - Log aggregators and shippers, metrics aggregators and shippers, and various other operations that you need to have running in your platform in order to maintain operational integrity.
- **Diagnostic Components** - Diagnose, troubleshoot, and improve system performance.

Principles of Microservices

- Scalability
- Availability
- Resiliency
- Independent, autonomous
- Decentralized governance
- Failure isolation
- Auto-Provisioning
- Continuous delivery through DevOps

Categories of Microservices Design Pattern

- Decomposition Patterns
  - Decompose by Business Capability
  - Decompose by Subdomain
  - Strangler
  - Side Car Pattern/Service Mesh
- Integration Patterns
  - API Gateway Pattern
  - Process Aggregator
  - Edge Pattern
- Database Patterns
  - Single Service Database
  - Shared Service
  - CQRS
- Observability Patterns
- Cross-Cutting Concern Patterns

Agenda

- Takeaway for Various Stakeholders
- History of Service Based Architectures
- About Microservices
- Microservices Core/Advanced Concepts
- **Solving Microservices Problems with Patterns**
- Microservices Architecture Choices

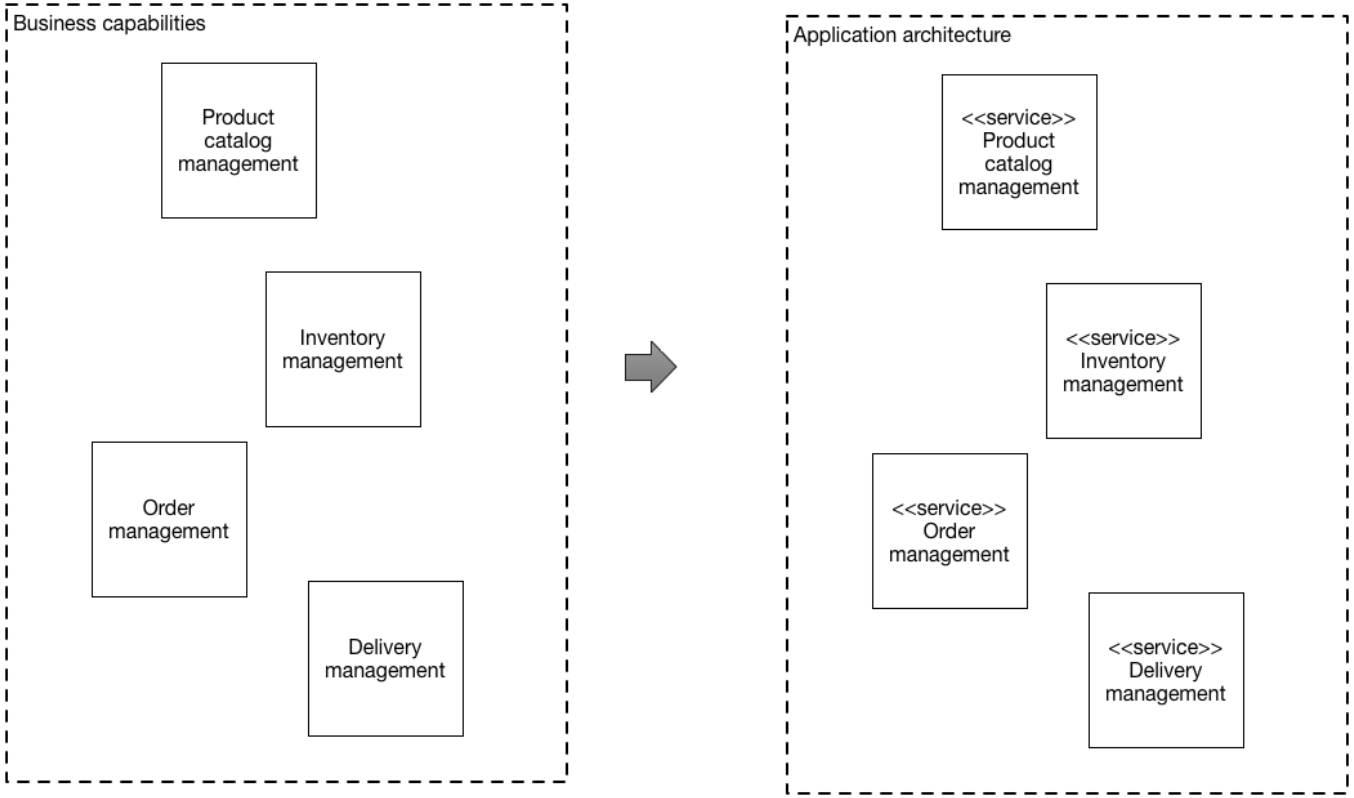
Decomposition Pattern – Decompose by Business Capability (1/3)

- Define services corresponding to business capabilities. A business capability is a concept from business architecture modeling. It is something that a business does in order to generate value. A business capability often corresponds to a business object, e.g.
  - Order Management is responsible for orders
  - Customer Management is responsible for customers
- Business capabilities are often organized into a multi-level hierarchy. For example, an enterprise application might have top-level categories such as:
  - Product/Service development
  - Product/Service delivery
  - Demand generation, etc.

Decomposition Pattern – Decompose by Business Capability (2/3)

Examples: The Business Capabilities of an Online Store Include:

- Product catalog management
- Inventory management
- Order management
- Delivery management



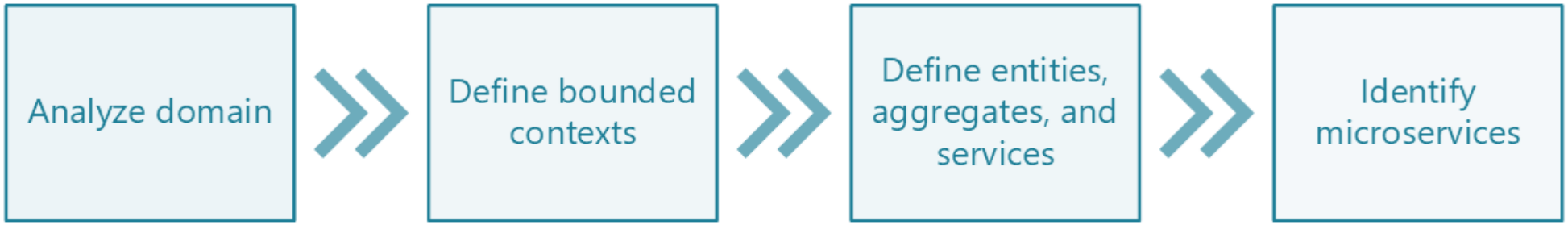
Decomposition Pattern – Decompose by Business Capability (3/3)

How to Identify Business Capabilities?

- Identifying business capabilities and hence services requires an understanding of the business. An organization's business capabilities are identified by analyzing the organization's purpose, structure, business processes, and areas of expertise. Bounded contexts are best identified using an iterative process. Good starting points for identifying business capabilities are:
  - Organization structure - different groups within an organization might correspond to business capabilities or business capability groups.
  - High-level domain model - business capabilities often correspond to domain objects.

Domain Driven Design

- Domain-driven design (DDD) provides a framework that can get you most of the way to a set of well-designed microservices. DDD has two distinct phases, strategic and tactical.
- In strategic DDD, you are defining the large-scale structure of the system. Strategic DDD helps to ensure that your architecture remains focused on business capabilities.
- Tactical DDD provides a set of design patterns that you can use to create the domain model. These patterns include entities, aggregates, and domain services. These tactical patterns will help you to design microservices that are both loosely coupled and cohesive.



Using Domain Analysis to Model Microservices

Scenario: Drone Delivery

- Fabrikam, Inc. is starting a drone delivery service. The company manages a fleet of drone aircraft. Businesses register with the service, and users can request a drone to pick up goods for delivery. When a customer schedules a pickup, a backend system assigns a drone and notifies the user with an estimated delivery time. While the delivery is in progress, the customer can track the location of the drone, with a continuously updated ETA.
- This scenario involves a fairly complicated domain. Some of the business concerns include scheduling drones, tracking package, managing user accounts, and storing and analyzing historical data. Moreover, Fabrikam wants to get to market quickly and then iterate quickly, adding new functionality and capabilities. The application needs to operate at cloud scale, with a high service level objective(SLO). Fabrikam also expects that different parts of the system will have very different requirements for data storage and querying. All of these considerations lead Fabrikam to choose a microservices architecture for the Drone Delivery application.

Analyze the Domain

- DDD approach will help you to design microservices so that every service forms a natural fit to a functional business requirement.
- You need a bird's eye view of the system that you are creating. DDD starts by modeling the business domain and creating a domain model. The domain model is an abstract model of the business domain. It distills and organizes domain knowledge and provides a common language for developers and domain experts.
- Start by mapping all of the business functions and their connections. This will likely be a collaborative effort that involves domain experts, software architects, and other stakeholders. -Sketch a diagram.
- As you fill in the diagram, you may start to identify discrete subdomains. Which functions are closely related? Which functions are core to the business, and which provide ancillary services? What is the dependency graph?
- During this initial phase, you aren't concerned with technologies or implementation details. That said, you should note the place where the application will need to integrate with external systems, such as CRM, payment processing, or billing systems.

Example: Drone Delivery Application

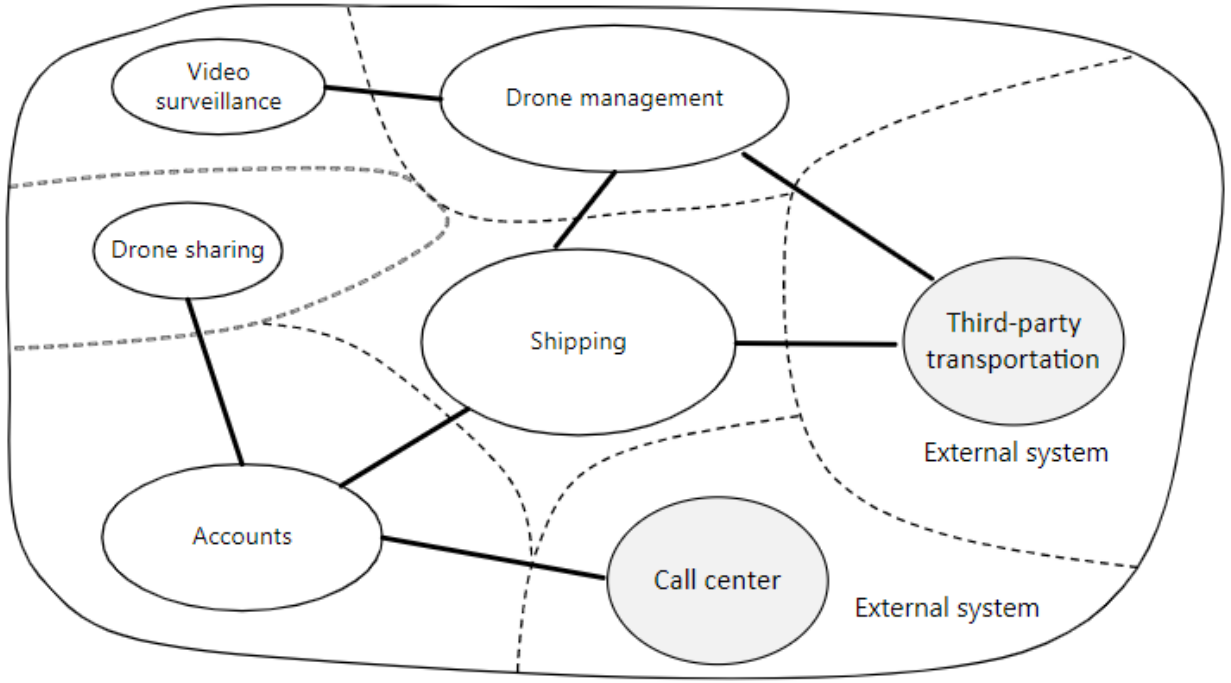
Drone Delivery Domain – Initial Analysis

- **Shipping** is placed in the center of the diagram, because it's core to the business. Everything else in the diagram exists to enable this functionality.
- **Drone management** is also core to the business. Functionality that is closely related to drone management includes **drone repair** and using **predictive analysis** to predict when drones need servicing and maintenance.
- **ETA analysis** provides time estimates for pickup and delivery.
- **Third-party transportation** will enable the application to schedule alternative transportation methods if a package cannot be shipped entirely by drone.
- **Drone sharing** is a possible extension of the core business. The company may have excess drone capacity during certain hours and could rent out drones that would otherwise be idle. This feature will not be in the initial release.
- **Video surveillance** is another area that the company might expand into later.
- **User accounts, invoicing,** and **call center** are subdomains that support the core business.

Bounded Context: Drone Delivery Application (1/2)

- **Repair and Predictive Analysis**: Subsystems that handle drone repair and predictive analysis will need to represent many physical characteristics of drones, such as their maintenance history, mileage, age, model number, performance characteristics, and so on.
- **Scheduling**: When it's time to schedule a delivery, we don't care about those things. The scheduling subsystem only needs to know whether a drone is available, and the ETA for pickup and delivery.
- If we tried to create a single model for both subsystems, it would be unnecessarily complex. It would also become harder for the model to evolve over time, because any changes will need to satisfy multiple teams working on separate subsystems.
- Therefore, it's often better to design separate models that represent the same real-world entity (in this case, a drone) in two different contexts. Each model contains only the features and attributes that are relevant within its particular context.
- This is where the DDD concept of bounded contexts comes into play. A bounded context is simply the boundary within a domain where a particular domain model applies.

Bounded Context: Drone Delivery Application (2/2)



Looking at the previous diagram, we can group functionality according to whether various functions will share a single domain model.

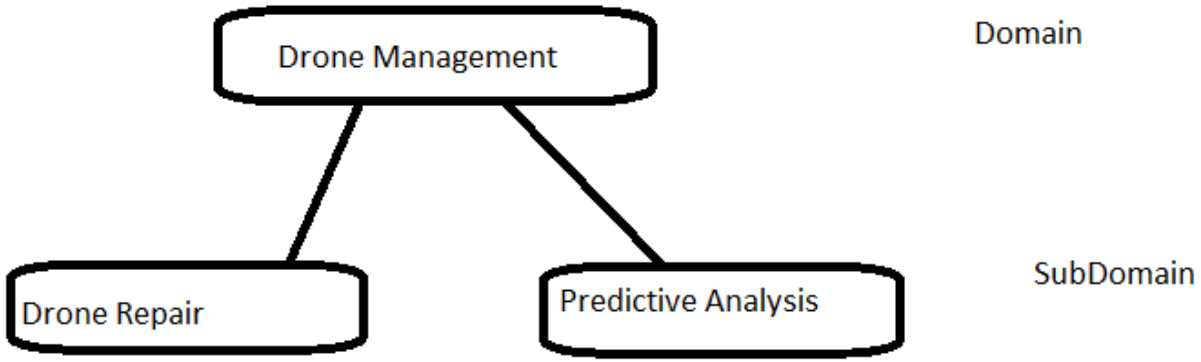
Decomposition Pattern – Decompose by Subdomain

Problem:

- Decomposing an application using business capabilities might be a good start, but you will come across so-called "God Classes" which will not be easy to decompose.
- These classes will be common among multiple services. For example, the Order class will be used in Order Management, Order Taking, Order Delivery, etc. How do we decompose them?

Solution:

- For the "God Classes" issue, DDD (Domain-Driven Design) comes to the rescue. It uses subdomains and bounded context concepts to solve this problem.
- DDD breaks the whole domain model created for the enterprise into subdomains. Each subdomain will have a model, and the scope of that model will be called the bounded context.
- Each microservice will be developed around the bounded context.



Decomposition Pattern – Strangler Pattern (1/3)

Strangler Pattern:

- Incrementally migrate a legacy system by gradually replacing specific pieces of functionality with new applications and services. As features from the legacy system are replaced, the new system eventually replaces all of the old system's features, strangling the old system and allowing you to decommission it.

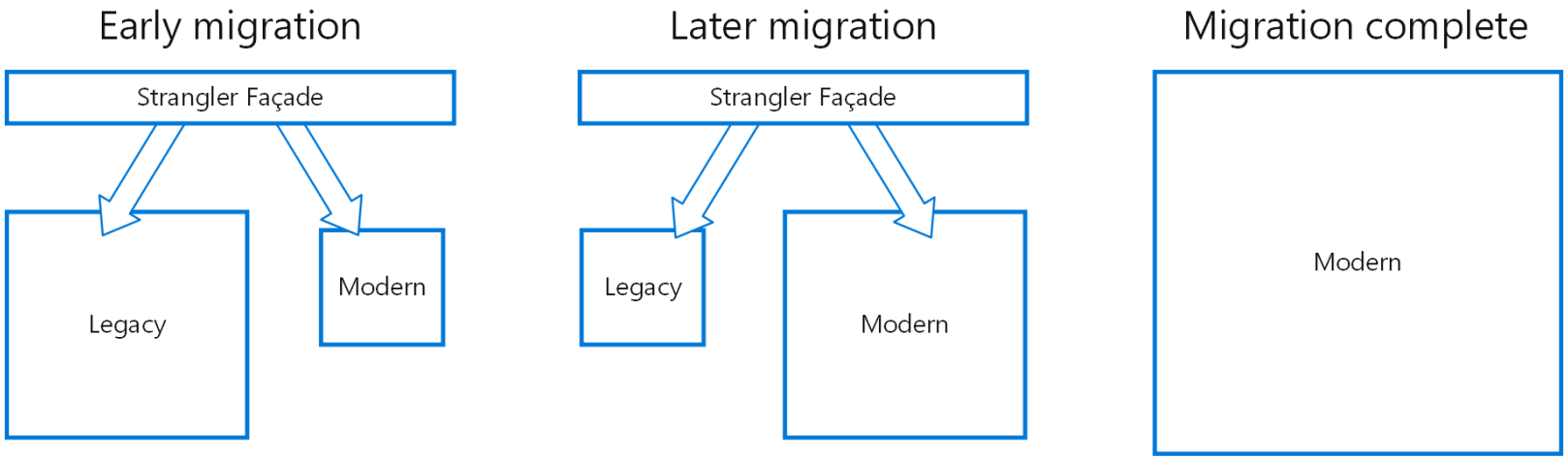
Problem:

- So far, the design patterns we talked about were decomposing applications for greenfield, but 80% of the work we do is with brownfield applications, which are big, monolithic applications. Applying all the above design patterns to them will be difficult because breaking them into smaller pieces at the same time it's being used live is a big task.
- Completely replacing a complex system can be a huge undertaking. Often, you will need a gradual migration to a new system, while keeping the old system to handle features that haven't been migrated yet. However, running two separate versions of an application means that clients must know where features are located. Every time a feature or service is migrated, clients need to be updated to point to the new location.

Decomposition Pattern – Strangler Pattern (2/3)

Solution:

- Incrementally replace specific pieces of functionality with new applications and services. Create a façade that intercepts requests going to the backend legacy system. The façade routes these requests either to the legacy application or the new services. Existing features can be migrated to the new system gradually, and consumers can continue using the same interface, unaware that any migration has taken place.



- This pattern helps to minimize risk from the migration and spread the development effort over time. With the façade safely routing users to the correct application, you can add functionality to the new system at whatever pace you like, while ensuring the legacy application continues to function. Over time, as features are migrated to the new system, the legacy system is eventually "strangled" and is no longer necessary. Once this process is complete, the legacy system can safely be retired.

Decomposition Pattern – Strangler Pattern (3/3)

Issues and Considerations:

- Consider how to handle services and data stores that are potentially used by both new and legacy systems. Make sure both can access these resources side-by-side.
- Structure new applications and services in a way that they can easily be intercepted and replaced in future strangler migrations.
- At some point, when the migration is complete, the strangler façade will either go away or evolve into an adaptor for legacy clients.
- Make sure the façade keeps up with the migration.
- Make sure the façade doesn't become a single point of failure or a performance bottleneck.

Decomposition Pattern – Side Car (1/3)

- **Service Mesh architecture** has been a game changer. It shifts the complexity associated with the Microservice architecture to a separate infrastructure layer and provides lot of functionalities like Load Balancing, Service Discovery, Traffic Management, Circuit Breaking, Telemetry, Fault Injection and more.

What is a Sidecar Pattern?

- Segregating the functionalities of an application into a separate process can be viewed as a Sidecar pattern.
- The Sidecar design pattern allows you to add a number of capabilities to your application without the need of additional configuration code for 3rd party components.
- As a sidecar is attached to a motorcycle, similarly in software architecture a sidecar is attached to a parent application and extends/enhances its functionalities. A Sidecar is loosely coupled with the main application.

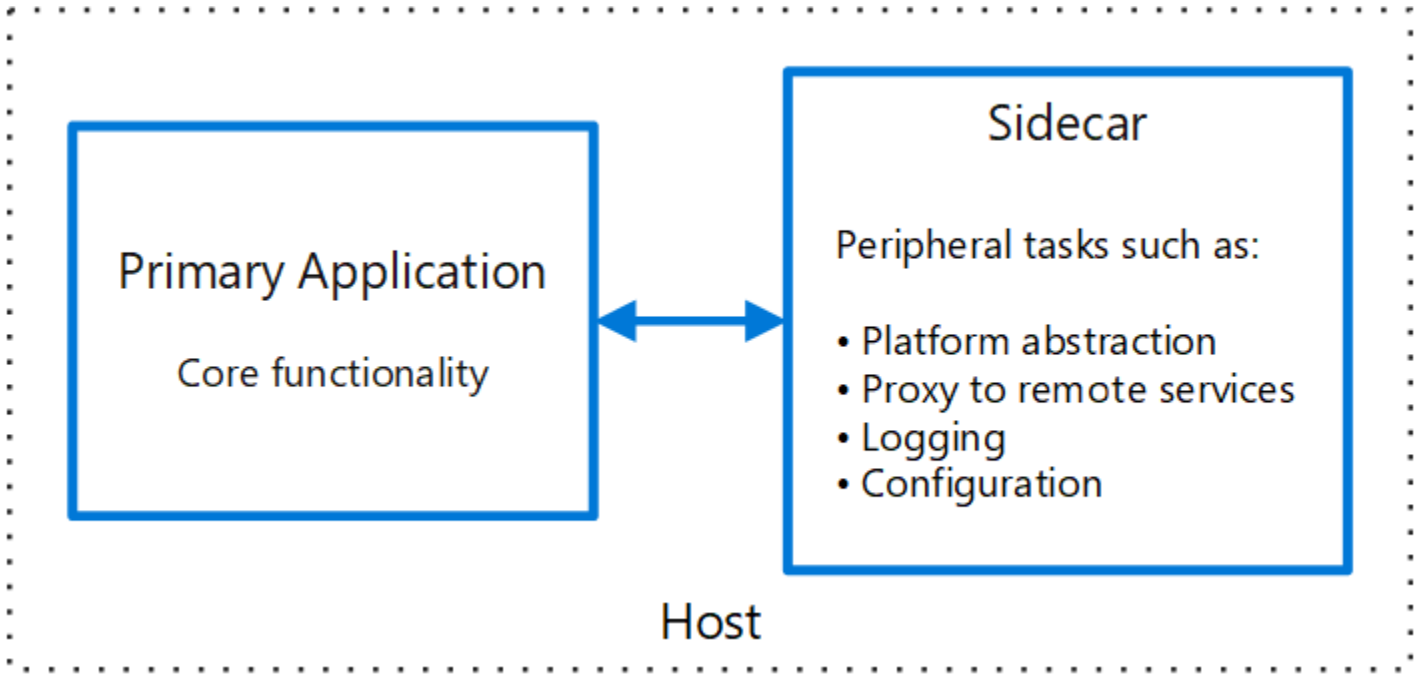
Problem Statement:

- Consider that you have 6 microservices talking with each other in order to determine the cost of a package.
- But think again, is this not redundant? Does it not increase the overall complexity of your application?
- What happens if your applications are written in different languages - how do you incorporate the 3rd party libraries which are generally specific to a language like .Net, Java, Python etc.

Decomposition Pattern – Side Car (2/3)

Solution:

- Co-locate a cohesive set of tasks with the primary application, but place them inside their own process or container, providing a homogeneous interface for platform services across languages.
- A sidecar service is not necessarily part of the application but is connected to it. It goes wherever the parent application goes. Sidecars are supporting processes or services that are deployed with the primary application.



Decomposition Pattern – Side Car (3/3)

Benefits:

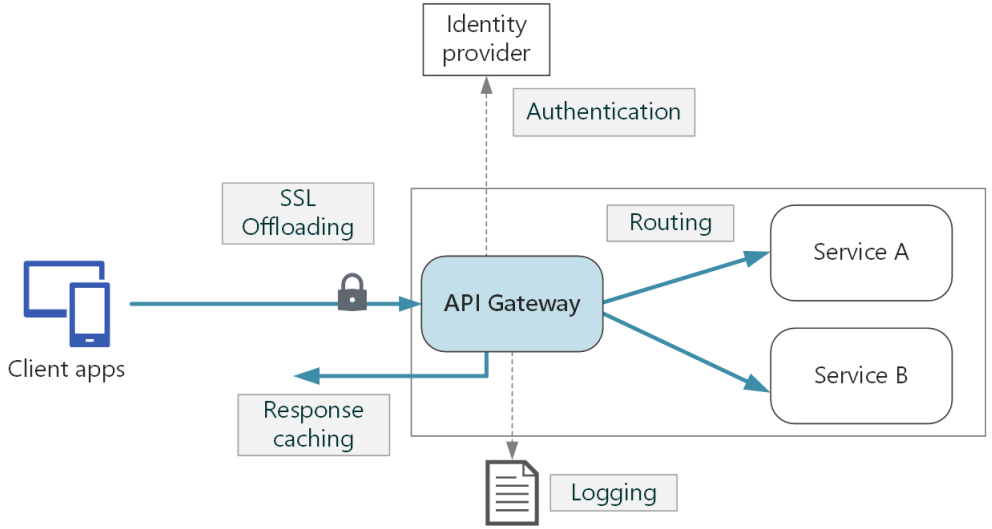
- A sidecar is independent from its primary application in terms of runtime environment and programming language, so you don't need to develop one sidecar per language.
- The sidecar can access the same resources as the primary application. For example, a sidecar can monitor system resources used by both the sidecar and the primary application.
- Because of its proximity to the primary application, there’s no significant latency when communicating between them.
- Even for applications that don’t provide an extensibility mechanism, you can use a sidecar to extend functionality by attaching it as its own process in the same host or sub-container as the primary application.

Issues and Considerations:

- Consider the deployment and packaging format you will use to deploy services, processes, or containers. Containers are particularly well suited to the sidecar pattern.
- When designing a sidecar service, carefully decide on the interprocess communication mechanism. Try to use language- or framework-agnostic technologies unless performance requirements make that impractical.
- Before putting functionality into a sidecar, consider whether it would work better as a separate service or a more traditional daemon.
- Also consider whether the functionality could be implemented as a library or using a traditional extension mechanism. Language-specific libraries may have a deeper level of integration and less network overhead.

Integration Pattern – API Gateway (1/3)

- When an application is broken down to smaller microservices, a client might interact with more than one front-end service. Given this fact, how does a client know what endpoints to call?
  - What happens when new services are introduced, or existing services are refactored?
  - How do services handle SSL termination, authentication, and other concerns?
  - How to call multiple microservices abstracting producer information?
  - Different consumers might need a different format of the responses from reusable microservices. Who will do the data transformation or field manipulation?
  - How to handle different type of Protocols some of which might not be supported by producer microservice? An API gateway can help to address these challenges.



Integration Pattern – API Gateway (2/3)

What is API Gateway?

- An API gateway sits between clients and services. It acts as a reverse proxy, routing requests from clients to services.
- It may also perform various cross-cutting tasks such as authentication, SSL termination, and rate limiting.
- If you don't deploy a gateway, clients must send requests directly to front-end services. However, there are some potential problems with exposing services directly to clients:
  - It can result in complex client code. The client must keep track of multiple endpoints, and handle failures in a resilient way.
  - It creates coupling between the client and the backend. The client needs to know how the individual services are decomposed. That makes it harder to maintain the client and also harder to refactor services.
  - A single operation might require calls to multiple services. That can result in multiple network round trips between the client and the server, adding significant latency.
  - Each public-facing service must handle concerns such as authentication, SSL, and client rate limiting.
  - Services must expose a client-friendly protocol such as HTTP or WebSocket. This limits the choice of communication protocols.
  - Services with public endpoints are a potential attack surface and must be hardened.

Integration Pattern – API Gateway (3/3)

Strategy:

- Define contracts.
- Expose APIs for those contracts, client focused.
- Adhere to strict version control and passive changes only.
- Implement the gateway to call your services and your clients to call the gateway.

Integration Pattern – Process Aggregator Pattern (1/3)

Problem:

- We have talked about resolving the aggregating data problem in the API Gateway Pattern. However, we will talk about it here holistically.
- When breaking the business functionality into several smaller logical pieces of code, it becomes necessary to think about how to collaborate the data returned by each service.
- This responsibility cannot be left with the consumer, as then it might need to understand the internal implementation of the producer application.

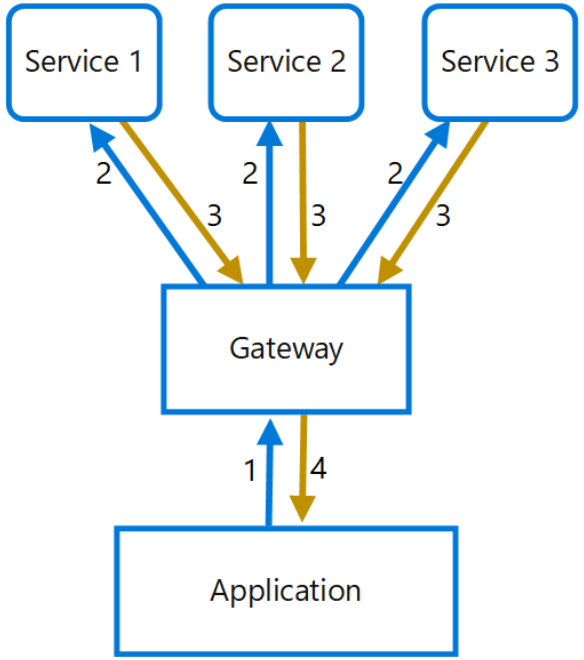
Solution:

- The Aggregator pattern helps to address this. It talks about how we can aggregate the data from different services and then send the final response to the consumer. This can be done in two ways:
  - A composite microservice will make calls to all the required microservices, consolidate the data, and transform the data before sending back.
  - An API Gateway can also partition the request to multiple microservices and aggregate the data before sending it to the consumer.
- It is recommended if any business logic is to be applied, then choose a composite microservice. Otherwise, the API Gateway is the established solution.

Integration Pattern – Process Aggregator Pattern (2/3)

Problem:

- We have talked about resolving the aggregating data problem in the API Gateway Pattern. However, we will talk about it here holistically.
- When breaking the business functionality into several smaller logical pieces of code, it becomes necessary to think about how to collaborate the data returned by each service.
- This responsibility cannot be left with the consumer, as then it might need to understand the internal implementation of the producer application.



Integration Pattern – Process Aggregator Pattern (3/3)

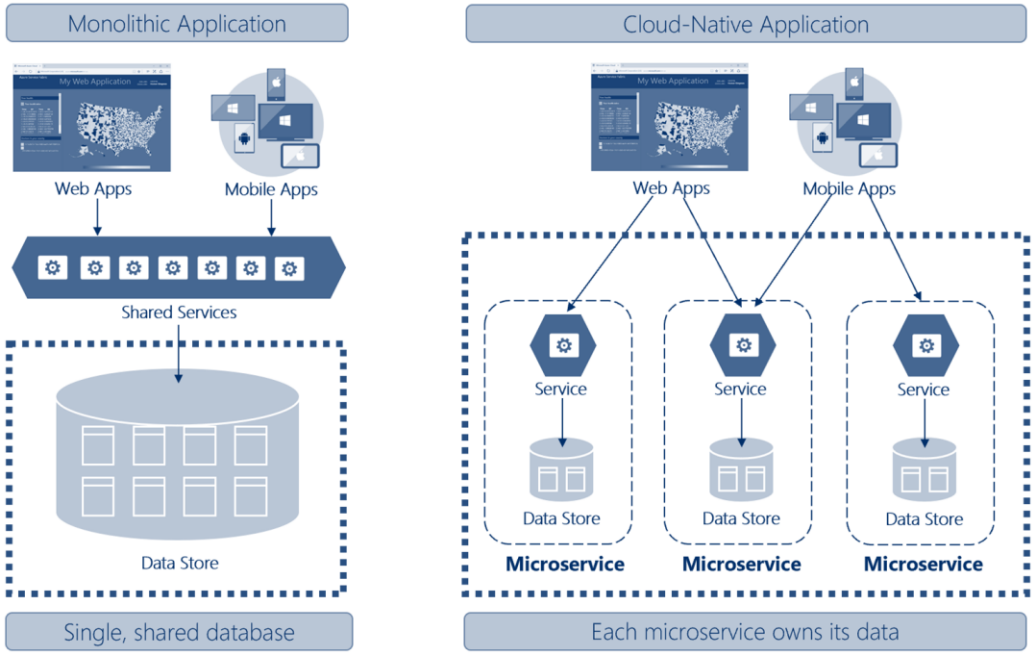
Aggregator Design:

- Determine the business processes.
- Determine the processing rules.
- Design a consolidated model.
- Design an API for the actions on that model.
- Wire the service and implement the internal processing.

Data Pattern – Database Per Microservice

Problem:

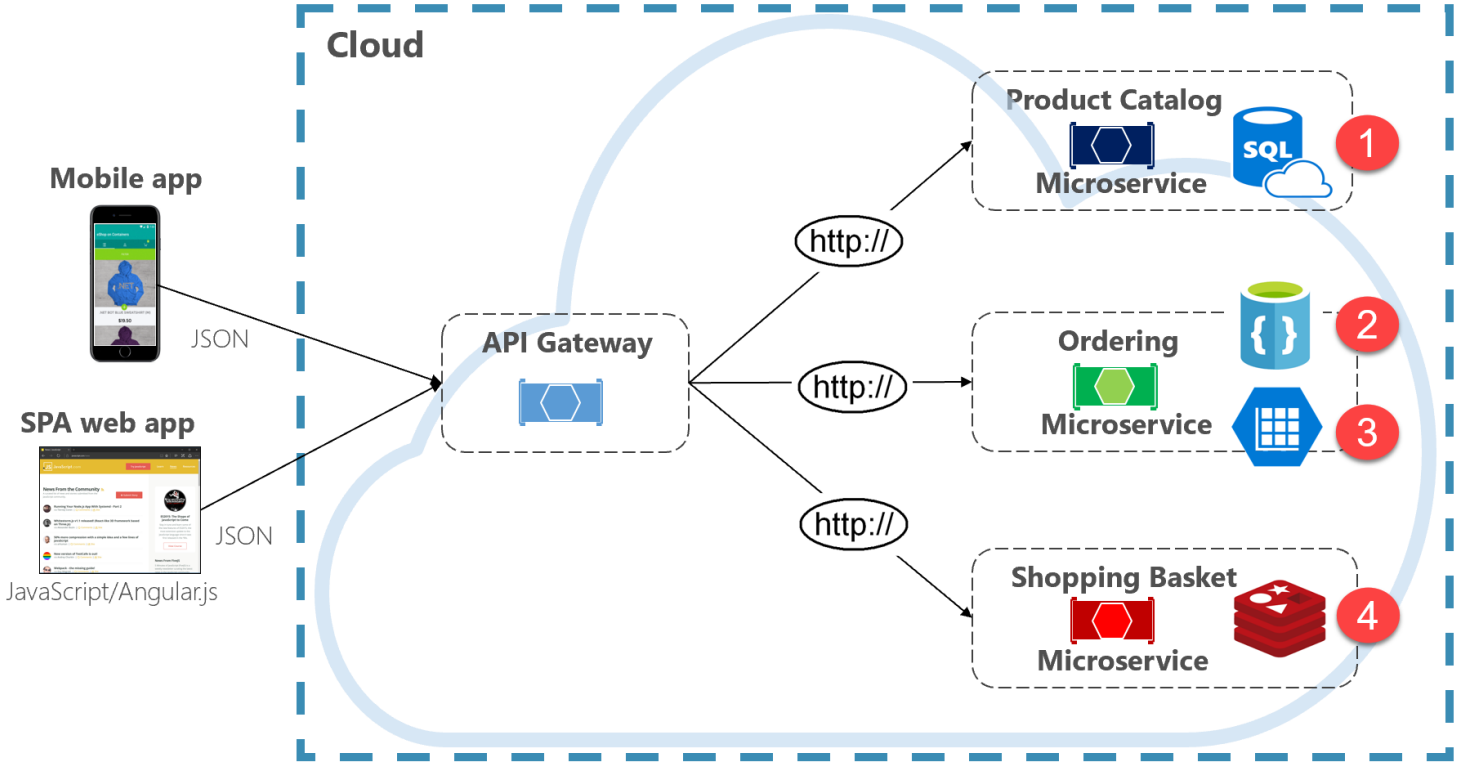
- There is a problem of how to define database architecture for microservices. Following are the concerns to be addressed:
  - Services must be loosely coupled. They can be developed, deployed, and scaled independently.
  - Business transactions may enforce invariants that span multiple services.
  - Some business transactions need to query data that is owned by multiple services.
  - Databases must sometimes be replicated and sharded in order to scale.
  - Different services have different data storage requirements.



Data Pattern – Database Per Microservice : Why? (1/2)

- Database per microservice provides many benefits, especially for systems that must evolve rapidly and support massive scale. With this model:
  - Domain data is encapsulated within the service.
  - Data schema can evolve without directly impacting other services.
  - Each data store can independently scale.
  - A data store failure in one service won't directly impact other services.
  - Segregating data also enables each microservice to implement the data store type that is best optimized for its workload, storage needs, and read/write patterns. Choices include relational, document, key-value, and even graph-based data stores.

Data Pattern – Database Per Microservice : Why? (2/2)

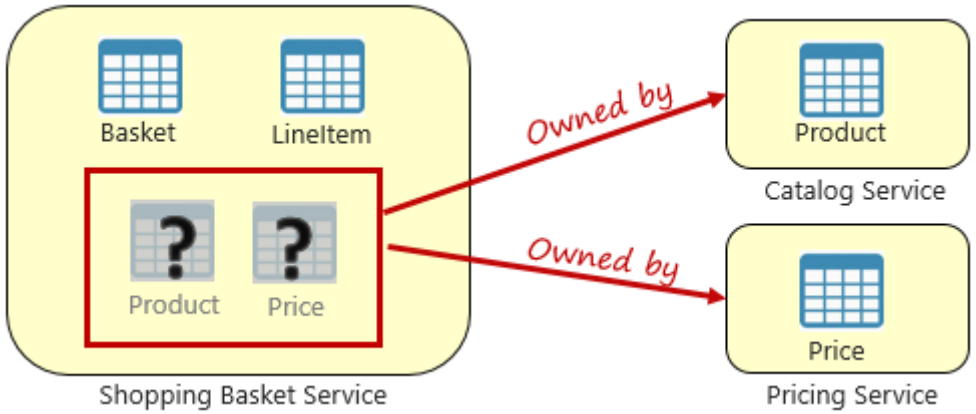


- The product catalog microservice consumes a relational database to accommodate the rich relational structure of its underlying data.
- The shopping cart microservice consumes a distributed cache that supports its simple, key-value data store.
- The ordering microservice consumes both a NoSql document database for write operations along with a highly denormalized key/value store to accommodate high-volumes of read operations.

Data Pattern – Database Per Microservice : Challenges

Cross-service Queries:

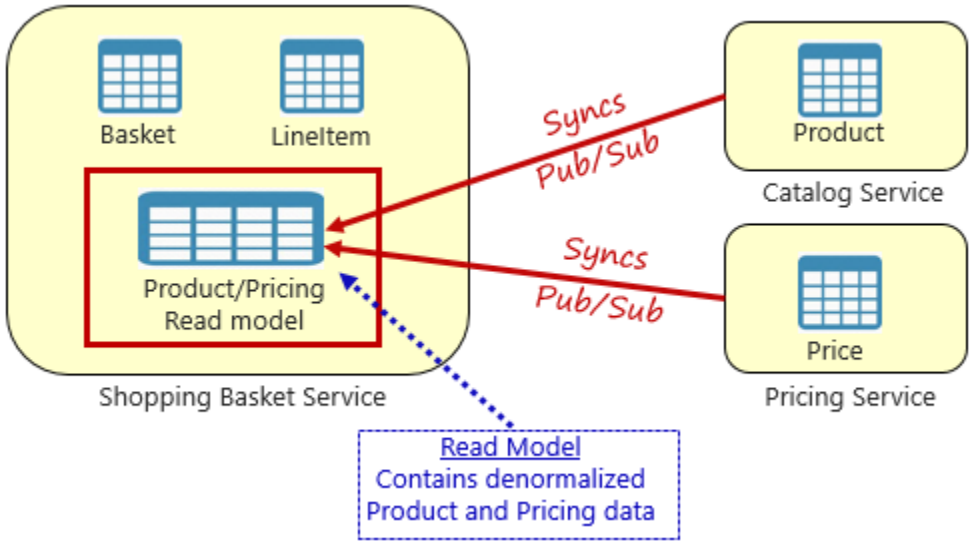
- While microservices are independent and focus on specific functional capabilities, like inventory, shipping, or ordering, they frequently require integration with other microservices. Often the integration involves one microservice querying another for data.



- We could implement a request-reply pattern with separate inbound and outbound queues for each service. However, this pattern is complicated and requires plumbing to correlate request and response messages.
- While it does decouple the backend microservice calls, the calling service must still synchronously wait for the call to complete. Network congestion, transient faults, or an overloaded microservice and can result in long-running and even failed operations.

Data Pattern – Database Per Microservice : Solution

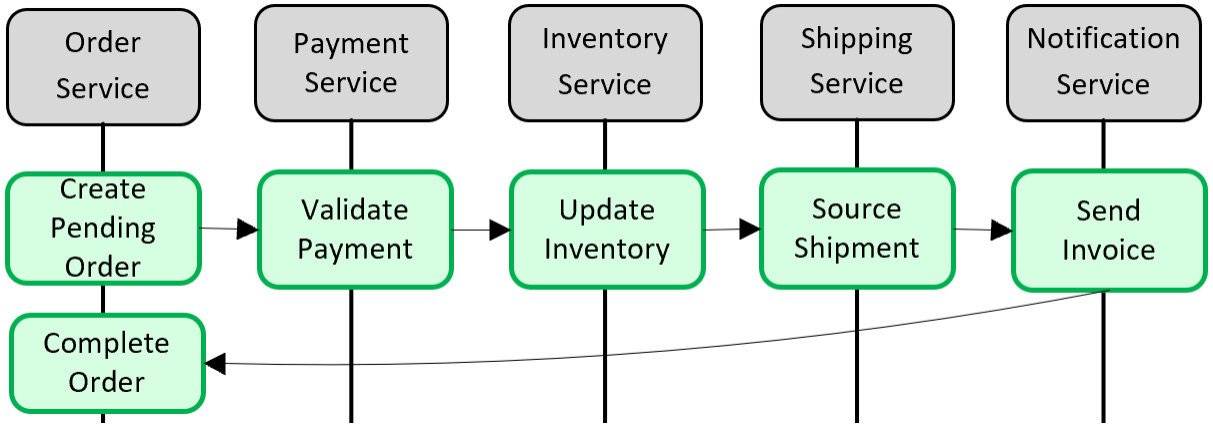
- Instead, a widely accepted pattern for removing cross-service dependencies is the Materialized View Pattern.



- With this pattern, you place a local data table (known as a read model) in the shopping basket service. This table contains a denormalized copy of the data needed from the product and pricing microservices. Copying the data directly into the shopping basket microservice eliminates the need for expensive cross-service calls. With the data local to the service, you improve the service's response time and reliability.

Data Pattern – Database Per Microservice : Distributed Transactions (1/2)

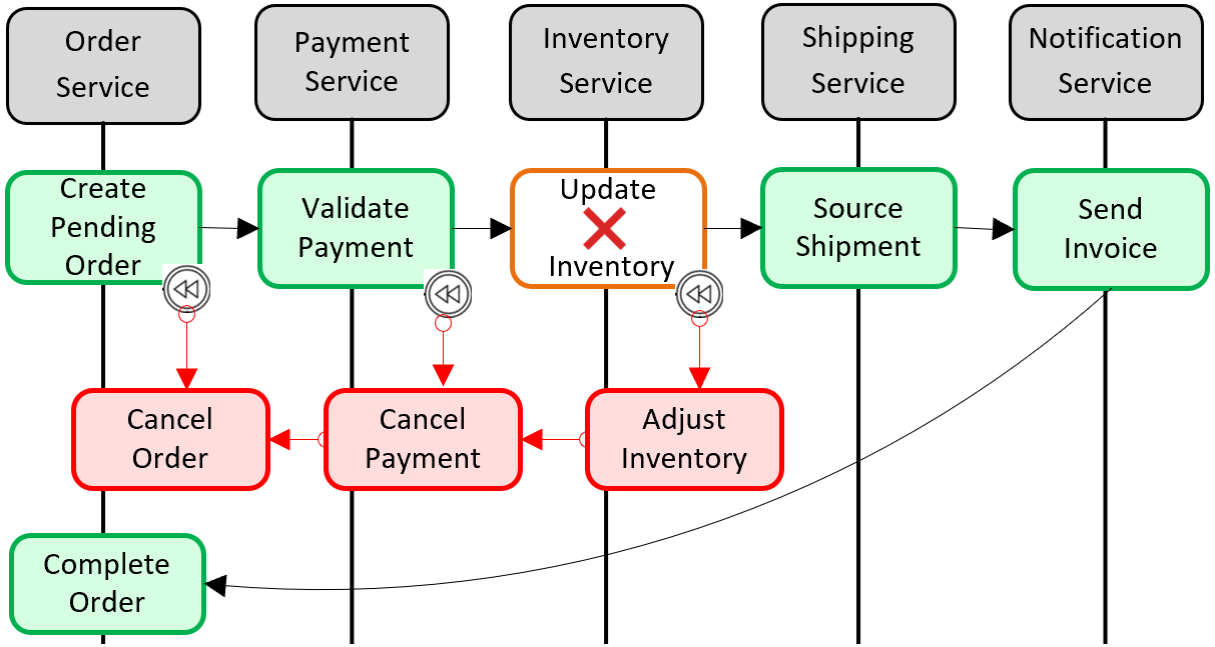
- While querying data across microservices is difficult, implementing a transaction across several microservices is even more complex. The inherent challenge of maintaining data consistency across independent data sources in different microservices can't be understated.
- The lack of distributed transactions in cloud-native applications means that you must manage distributed transactions programmatically. You move from a world of immediate consistency to that of eventual consistency.



Data Pattern – Database Per Microservice : Distributed Transactions (2/2)

Solution:

- You must construct this distributed transaction programmatically.
- A popular pattern for adding distributed transactional support is the Saga pattern. It's implemented by grouping local transactions together programmatically and sequentially invoking each one. If any of the local transactions fail, the Saga aborts the operation and invokes a set of compensating transactions.
- The compensating transactions undo the changes made by the preceding local transactions and restore data consistency.



Data Pattern – Shared Service Database (1/2)

Problem:

- We have talked about one database per service being ideal for microservices, but that is possible when the application is greenfield and to be developed with DDD. But if the application is a monolith and trying to break into microservices, denormalization is not that easy. What is the suitable architecture in that case?

Solution:

- A shared database per service is not ideal, but that is the working solution for the above scenario.
- Most people consider this an anti-pattern for microservices, but for brownfield applications, this is a good start to break the application into smaller logical pieces.
- This should not be applied for greenfield applications. In this pattern, one database can be aligned with more than one microservice, but it has to be restricted to 2-3 maximum, otherwise scaling, autonomy, and independence will be challenging to execute.

Data Pattern – Shared Service Database (2/2)

Need:

- Some business transactions must enforce invariants that span multiple services. For example, the Place Order use case must verify that a new Order will not exceed the customer's credit limit. Other business transactions, must update data owned by multiple services.
- Some business transactions need to query data that is owned by multiple services. For example, the View Available Credit use must query the Customer to find the creditLimit and Orders to calculate the total amount of the open orders.
- Some queries must join data that is owned by multiple services. For example, finding customers in a particular region and their recent orders requires a join between customers and orders.
- Databases must sometimes be replicated and sharded in order to scale. See the Scale Cube.
- Different services have different data storage requirements. For some services, a relational database is the best choice. Other services might need a NoSQL database such as MongoDB, which is good at storing complex, unstructured data, or Neo4J, which is designed to efficiently store and query graph data.

Data Pattern – Command Query Responsibility Segregation (1/2)

High Volume Data:

- Large cloud-native applications often support high-volume data requirements. In these scenarios, traditional data storage techniques can cause bottlenecks. For complex systems that deploy on a large scale, both Command and Query Responsibility Segregation (CQRS) and Event Sourcing may improve application performance.

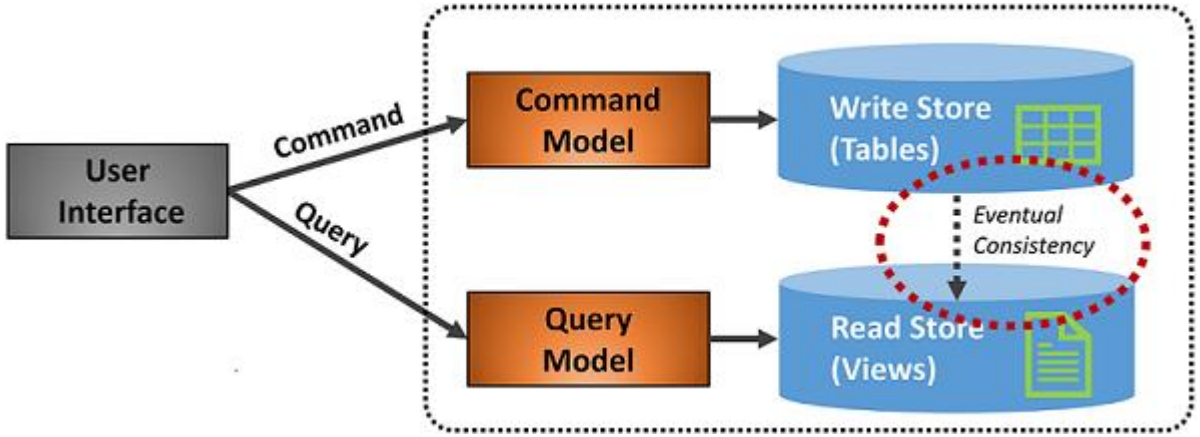
CQRS:

- It is an architectural pattern that can help maximize performance, scalability, and security. The pattern separates operations that read data from those operations that write data.
- To improve performance, the read operation could query against a highly denormalized representation of the data to avoid expensive repetitive table joins and table locks.
- The write operation, known as a command, would update against a fully normalized representation of the data that would guarantee consistency. You then need to implement a mechanism to keep both representations in sync. Typically, whenever the write table is modified, it publishes an event that replicates the modification to the read table.

Data Pattern – Command Query Responsibility Segregation (2/2)

High Volume Data:

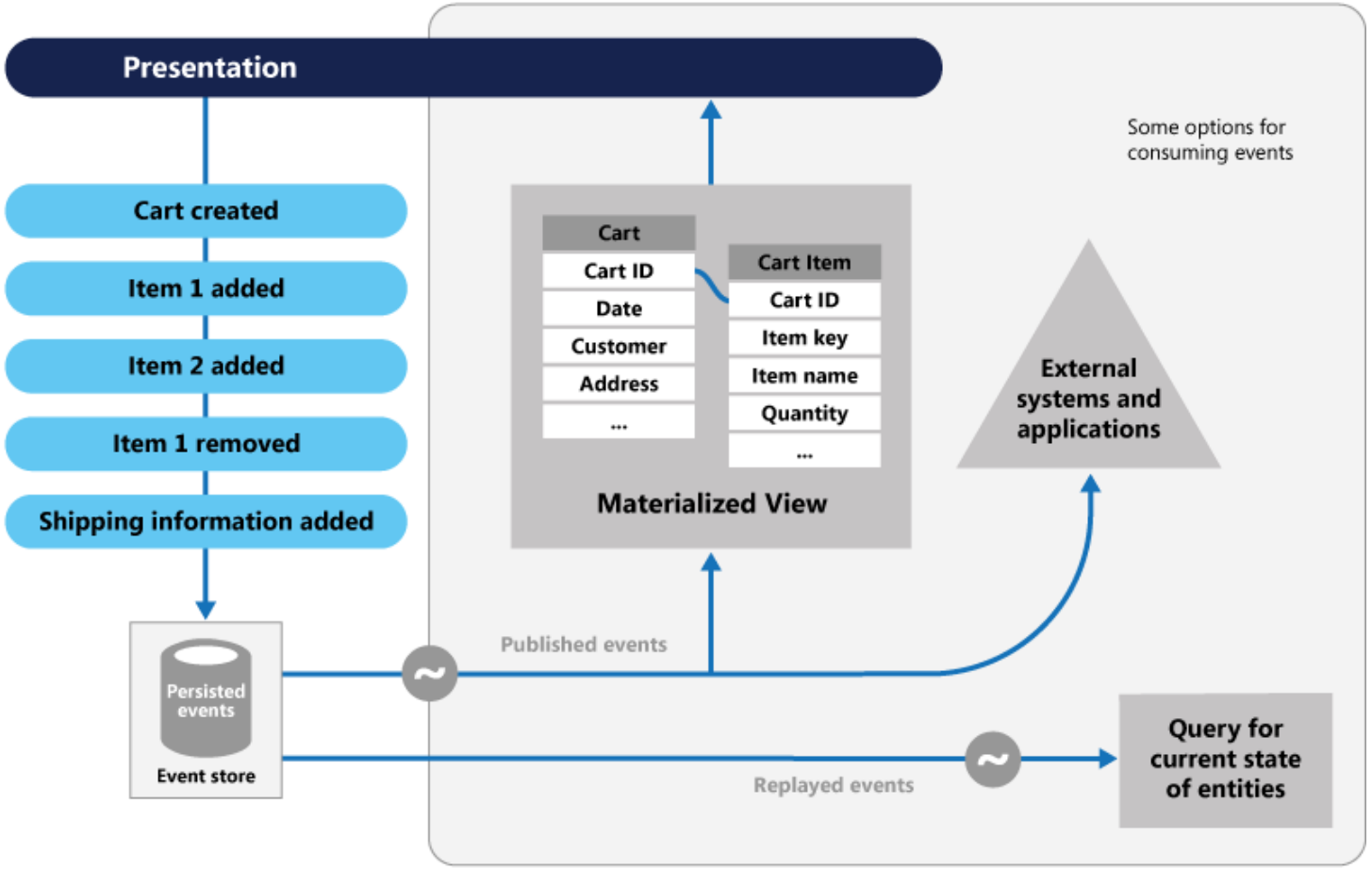
- Each data write operation is saved to the write store and then propagated to the read store. Pay close attention to how the data propagation process operates on the principle of **eventual consistency**. The read model eventually synchronizes with the write model, but there may be some lag in the process.



Data Pattern – Event Sourcing (1/2)

- A system typically stores the current state of a data entity. If a user changes their phone number, for example, the customer record is updated with the new number. We always know the current state of a data entity, but each update overwrites the previous state.
- In most cases, this model works fine. In high volume systems, however, overhead from transactional locking and frequent update operations can impact database performance, responsiveness, and limit scalability.
- Event Sourcing takes a different approach to capturing data. Each operation that affects data is persisted to an event store. Instead of updating the state of a data record, we append each change to a sequential list of past events - similar to an accountant's ledger.
- The Event Store becomes the system of record for the data. It's used to propagate various materialized views within the bounded context of a microservice.

Data Pattern – Event Sourcing (2/2)



https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing

For this pattern, consider a data store that directly supports event sourcing. Azure Cosmos DB, MongoDB, Cassandra, CouchDB, and RavenDB are good candidates.

Observability Pattern – Log Aggregation (1/3)

Problem:

- Consider a use case where an application consists of multiple service instances that are running on multiple machines. Requests often span multiple service instances.
- Each service instance generates a log file in a standardized format. How can we understand the application behavior through logs for a particular request?

Solution:

- We need a centralized logging service that aggregates logs from each service instance.
- Users can search and analyze the logs. They can configure alerts that are triggered when certain messages appear in the logs.
- For example, PCF does have Loggeregator, which collects logs from each component (router, controller, diego, etc...) of the PCF platform along with applications. **AWS Cloud Watch** also does the same.

Observability Pattern – Log Aggregation (2/3)

Logging:

- Problem: You need to know what is going on.
- Logging is invaluable in operations.
- Logging must be consistent.
- Logging must be structured.
- Logging must share a taxonomy.

Observability Pattern – Log Aggregation (3/3)

Log Aggregation:

- Problem: Logs are everywhere.
- Aggregation of logs.
- Parsing of logs.
- Correlating of logs.
- Indexing of logs.

Observability Pattern – Performance Metrics

Problem:

- When the service portfolio increases due to microservice architecture, it becomes critical to keep a watch on the transactions so that patterns can be monitored, and alerts sent when an issue happens. How should we collect metrics to monitor application perfomance?

Solution:

- A metrics service is required to gather statistics about individual operations. It should aggregate the metrics of an application service, which provides reporting and alerting. There are two models for aggregating metrics:
  - Push - The service pushes metrics to the metrics service e.g. NewRelic, AppDynamics.
  - Pull - The metrics services pulls metrics from the service e.g. Prometheus.

Agenda

- Takeaway for Various Stakeholders
- History of Service Based Architectures
- About Microservices
- Microservices Core/Advanced Concepts
- Solving Microservices Problems with Patterns
- **Microservices Architecture Choices**

Saga Distributed Transactions Pattern (1/5)

- The saga design pattern is a way to manage data consistency across microservices in distributed transaction scenarios.
- A saga is a sequence of transactions that updates each service and publishes a message or event to trigger the next transaction step. If a step fails, the saga executes compensating transactions that counteract the preceding transactions.
- Transactions must be atomic, consistent, isolated, and durable (ACID). Transactions within a single service are ACID, but cross-service data consistency requires a cross-service transaction management strategy.
- In multiservices architectures:
  - Atomicity is an indivisible and irreducible set of operations that must all occur, or none occur.
  - Consistency means the transaction brings the data only from one valid state to another valid state.
  - Isolation guarantees that concurrent transactions produce the same data state that sequentially executed transactions would have produced.
  - Durability ensures that committed transactions remain committed even in case of system failure or power outage.

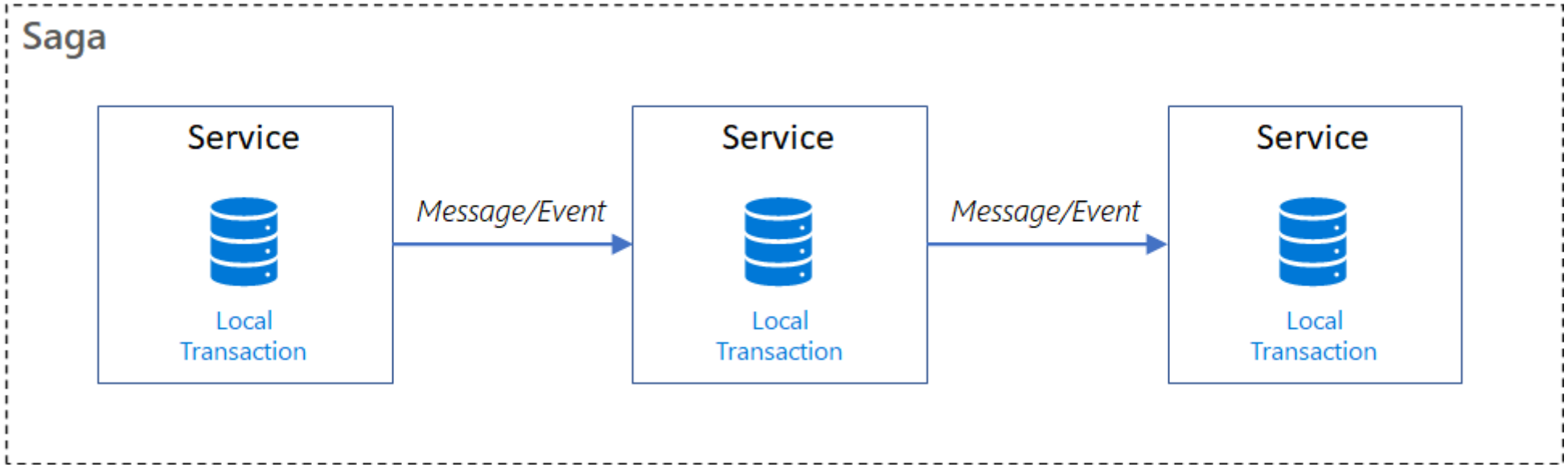
Saga Distributed Transactions Pattern (2/5)

- A database-per-microservice model provides many benefits for microservices architectures. Encapsulating domain data lets each service use its best data store type and schema, scale its own data store as necessary, and be insulated from other services' failures. However, ensuring data consistency across service-specific databases poses challenges.
- **Two Phase Commit** : Distributed transactions like the two-phase commit (2PC) protocol require all participants in a transaction to commit or roll back before the transaction can proceed. However, some participant implementations, such as NoSQL databases and message brokering, don't support this model.
- **InterProcess Communication**: Another distributed transaction limitation is interprocess communication (IPC) synchronicity and availability. Operating system-provided IPC allows separate processes to share data. For distributed transactions to commit, all participating services must be available, potentially reducing overall system availability. Architectural implementations with IPC or transaction limitations are candidates for the saga pattern.

Saga Distributed Transactions Pattern (3/5)

Solution:

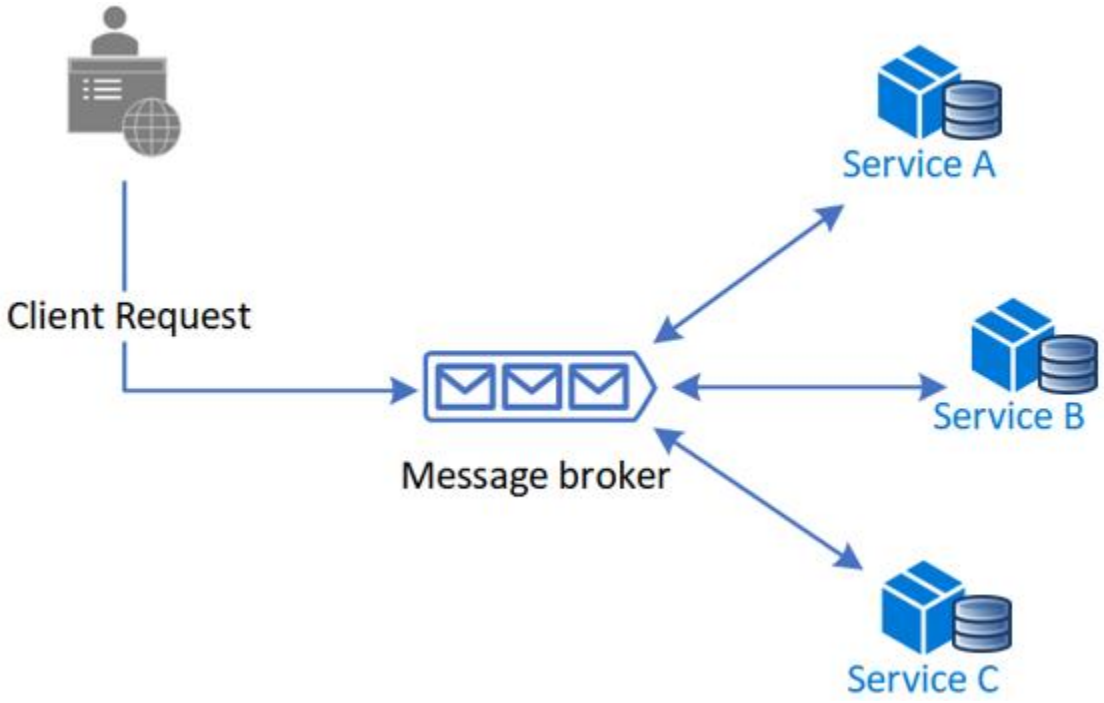
- The saga pattern provides transaction management using a sequence of local transactions. A local transaction is the atomic work effort performed by a saga participant.
- Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga. If a local transaction fails, the saga executes a series of compensating transactions that undo the changes that were made by the preceding local transactions.



Saga Distributed Transactions Pattern (4/5)

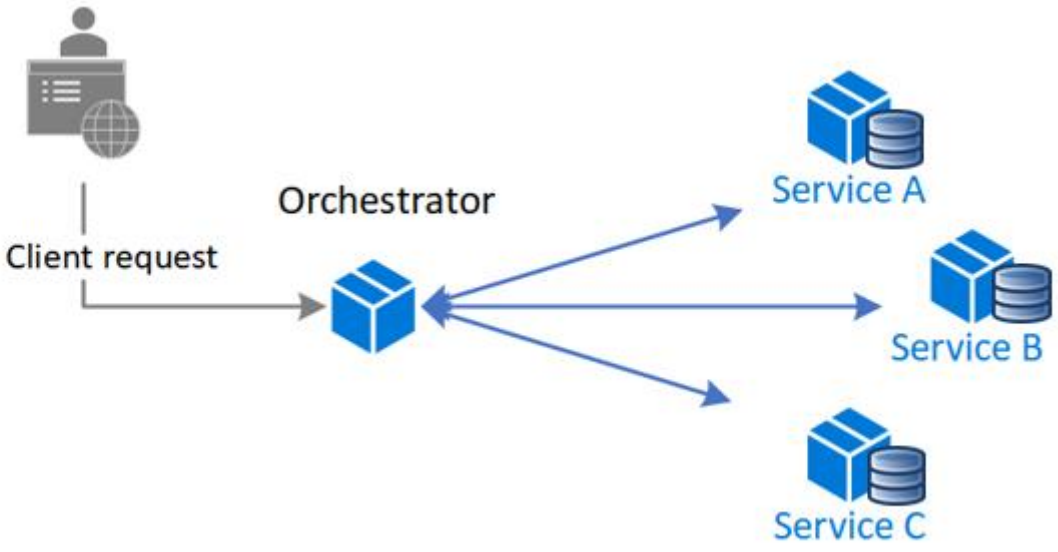
In Saga Pattern:

- There are two common saga implementation approaches, choreography and orchestration. Each approach has its own set of challenges and technologies to coordinate the workflow.
- **Choreography** is a way to coordinate sagas where participants exchange events without a centralized point of control. With choreography, each local transaction publishes domain events that trigger local transactions in other services.



Saga Distributed Transactions Pattern (5/5)

- **Orchestration** is a way to coordinate sagas where a centralized controller tells the saga participants what local transactions to execute.
- The saga orchestrator handles all the transactions and tells the participants which operation to perform based on events. The orchestrator executes saga requests, stores and interprets the states of each task, and handles failure recovery with compensating transactions.



Cross Cutting Concern Patterns – External Configuration

Problem:

- A service typically calls other services and databases as well. For each environment like dev, QA, UAT, prod, the endpoint URL or some configuration properties might be different. A change in any of those properties might require a re-build and re-deploy of the service. How do we avoid code modification for configuration changes?

Solution:

- Externalize all the configuration, including endpoint URLs and credentials. The application should load them either at startup or on the fly.
- **Spring Cloud config** server provides the option to externalize the properties to GitHub and load them as environment properties. These can be accessed by the application on startup or can be refreshed without a server restart.

Cross Cutting Concern Patterns – Service Discovery Pattern

Problem:

- When microservices come into the picture, we need to address a few issues in terms of calling services:
- With container technology, IP addresses are dynamically allocated to the service instances. Every time the address changes, a consumer service can break and need manual changes.
- Each service URL has to be remembered by the consumer and become tightly coupled.
- So how does the consumer or router know all the available service instances and locations?

Solution:

- A service registry needs to be created which will keep the metadata of each producer service. A service instance should register to the registry when starting and should de-register when shutting down. The consumer or router should query the registry and find out the location of the service.
- The registry also needs to do a health check of the producer service to ensure that only working instances of the services are available to be consumed through it. There are two types of service discovery: client-side and server-side. An example of client-side discovery is Netflix Eureka, and an example of server-side discovery is AWS ALB.

Cross Cutting Concern Patterns – Circuit Breaker Pattern

Problem:

- A service generally calls other services to retrieve data, and there is the chance that the downstream service may be down. There are two problems with this: first, the request will keep going to the down service, exhausting network resources and slowing performance. Second, the user experience will be bad and unpredictable. How do we avoid cascading service failures and handle failures gracefully?

Solution:

- The consumer should invoke a remote service via a proxy **that behaves in a similar fashion** to an electrical circuit breaker. When the number of consecutive failures crosses a threshold, the circuit breaker trips, and for the duration of a timeout period, all attempts to invoke the remote service will fail immediately. After the timeout expires the circuit breaker allows a limited number of test requests to pass through. If those requests succeed, the circuit breaker resumes normal operation. Otherwise, if there is a failure, the timeout period begins again.
- **Netflix Hystrix** is a good implementation of the circuit breaker pattern. It also helps you to define a fallback mechanism which can be used when the circuit breaker trips. That provides a better user experience.

Thank you!

Perspective from leaders across the healthcare industry

- On-Demand Webinars
- Reports & eBooks
- Workshops
- Success Stories
- Articles & Blogs
- News

Explore the Knowledge Hub >

CitiusTech

Powering the Future of Healthcare